# 35.3 WebChat 实现

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~160k tokens，输出 ~7k tokens（本节）

上一节剖析了控制台 UI 的整体架构。本节聚焦 WebChat——控制台中的聊天界面，它是用户直接与 AI 助手对话的主要入口。我们将分析 WebChat 作为 Gateway WebSocket 客户端的工作方式，以及聊天历史加载与消息发送的完整流程。

## 35.3.1 WebChat 作为 Gateway WS 客户端

### WebChat 的定位

WebChat 并不是一个独立的聊天服务——它本质上是 Gateway WebSocket 协议的一个客户端，与 Telegram、Discord 等渠道处于同一层次。区别在于：

| 特征 | WebChat | Telegram/Discord 等 |
| --- | --- | --- |
| 连接方式 | 浏览器 WebSocket | 服务端长连接/Webhook |
| 客户端模式 | `mode: "webchat"` | `mode: "channel"` |
| 角色 | `role: "operator"` | `role: "channel"` |
| 认证 | Ed25519 设备签名 + Token | API Token/OAuth |
| 会话选择 | 用户可自由切换 sessionKey | 由平台消息自动路由 |

WebChat 在连接 Gateway 时声明自己为 `openclaw-control-ui` 客户端，模式为 `webchat` ，角色为 `operator` ：

```
// ui/src/ui/gateway.ts（连接参数）
const params = {
  minProtocol: 3,
  maxProtocol: 3,
  client: {
    id: "openclaw-control-ui",
    mode: "webchat",
    platform: navigator.platform ?? "web",
  },
  role: "operator",
  scopes: ["operator.admin", "operator.approvals", "operator.pairing"],
};
```

`operator` 角色拥有完整的管理权限，包括管理员操作、执行审批、设备配对等。这意味着 WebChat 不仅能聊天，还能通过聊天命令（如 `/new` 、 `/stop` ）控制会话生命周期。

## 会话键（Session Key）

WebChat 的核心概念是 **sessionKey**——它决定了用户当前对话的上下文。sessionKey 是一个字符串标识符，例如 `"main"` 、 `"agent:researcher:main"` 等。用户可以在 UI 中自由切换会话。

```
// ui/src/ui/app-chat.ts
function resolveAgentIdForSession(host: ChatHost): string | null {
  // 从 sessionKey 解析 agentId（如 "agent:researcher:main" → "researcher"）
  const parsed = parseAgentSessionKey(host.sessionKey);
  if (parsed?.agentId) return parsed.agentId;
  // 退回到默认 Agent
  return snapshot?.sessionDefaults?.defaultAgentId || "main";
}
```

切换 sessionKey 时，UI 会清空当前的聊天草稿、流式内容、运行状态和工具流，然后重新加载对应会话的历史消息和头像：

```ts
// ui/src/ui/app-render.ts（切换会话的回调）
onSessionKeyChange: (next) => {
  state.sessionKey = next;
  state.chatMessage = "";
  state.chatAttachments = [];
  state.chatStream = null;
  state.chatRunId = null;
  state.chatQueue = [];
  state.resetToolStream();
  state.resetChatScroll();
  state.applySettings({
    ...state.settings,
    sessionKey: next,
    lastActiveSessionKey: next,
  });
  void loadChatHistory(state);
  void refreshChatAvatar(state);
},
```

## 事件驱动的聊天更新

发送消息后，UI 不会同步等待完整的回复。而是通过 Gateway 推送的 `chat` 事件实时更新界面：

```ts
// ui/src/ui/controllers/chat.ts
export type ChatEventPayload = {
  runId: string;          // 唯一标识本次对话轮次
  sessionKey: string;     // 目标会话
  state: "delta" | "final" | "aborted" | "error";
  message?: unknown;      // delta 时为部分响应
  errorMessage?: string;  // error 时的错误信息
};
```

事件状态机的转换：

```
发送消息 → chatRunId 设置
    │
    ├─ delta → 累积 chatStream（流式文本）
    ├─ delta → 累积 chatStream
    ├─ ...
    │
    ├─ final → 清空 stream/runId → 刷新历史
    ├─ aborted → 清空 stream/runId
    └─ error → 清空 stream/runId → 显示错误
```

`delta` 事件的处理有一个巧妙的设计——它不是追加文本，而是替换为更长的文本：

```
if (payload.state === "delta") {
  const next = extractText(payload.message);
  if (typeof next === "string") {
    const current = state.chatStream ?? "";
    // 只有当新文本更长时才更新（防止乱序）
    if (!current || next.length >= current.length) {
      state.chatStream = next;
    }
  }
}
```

这种“全量替换”模式避免了流式传输中 delta 包乱序导致的文本错乱问题。

## 跨运行事件过滤

`handleChatEvent` 会检查事件的 `runId` 是否匹配当前运行：

```
// 来自其他运行的 final 事件（如子 Agent announce）
if (payload.runId && state.chatRunId && payload.runId !== state.chatRunId) {
  if (payload.state === "final") return "final";  // 仍然触发历史刷新
  return null;  // 忽略其他状态
}
```

当一个 Agent 在后台运行完成后通过 announce 发送消息时，当前 WebChat 会收到 `final` 事件但 `runId` 不匹配。此时 UI 不更新流式状态，但仍然触发聊天历史刷新——确保用户能看到新消息。

# 35.3.2 聊天历史加载与消息发送

## 历史加载

聊天历史通过 `chat.history` RPC 方法从 Gateway 获取：

```
// ui/src/ui/controllers/chat.ts
export async function loadChatHistory(state: ChatState) {
  if (!state.client || !state.connected) return;
  state.chatLoading = true;
  try {
    const res = await state.client.request<{
      messages?: Array<unknown>;
      thinkingLevel?: string;
    }>("chat.history", {
      sessionKey: state.sessionKey,
      limit: 200,       // 最多加载 200 条历史消息
    });
    state.chatMessages = Array.isArray(res.messages) ? res.messages : [];
    state.chatThinkingLevel = res.thinkingLevel ?? null;
  } catch (err) {
    state.lastError = String(err);
  } finally {
    state.chatLoading = false;
  }
}
```

返回的 `thinkingLevel` 用于确定当前会话是否启用了推理/思考模式——如果 `reasoningLevel` 不是 `"off"` ，UI 会显示 thinking 相关的 token 输出。

## 消息渲染管线

加载的原始消息经过一条 **规范化** → **分组** → **渲染** 的管线：

```
原始消息数组（chatMessages）
        │
        ▼
  normalizeMessage()        ← 统一不同格式的消息结构
        │
        ▼
  normalizeRoleForGrouping()  ← 统一角色名（tool_result → tool）
        │
        ▼
  buildChatItems()          ← 构建 ChatItem 列表 + 注入流式/加载指示器
        │
        ▼
  groupMessages()           ← 按连续角色分组为 MessageGroup
        │
        ▼
  renderMessageGroup()      ← Lit 模板渲染：头像 + 气泡 + Markdown
```

**消息规范化**是关键一步。Gateway 返回的消息可能有多种格式（来自不同渠道或 API 版本）：

```ts
// ui/src/ui/chat/message-normalizer.ts
export function normalizeMessage(message: unknown): NormalizedMessage {
  const m = message as Record<string, unknown>;
  let role = typeof m.role === "string" ? m.role : "unknown";

  // 检测工具消息的多种形态
  const hasToolId = typeof m.toolCallId === "string" || typeof m.tool_call_id === "stri
  const hasToolName = typeof m.toolName === "string" || typeof m.tool_name === "string"
  if (hasToolId || hasToolName) role = "toolResult";

  // 统一内容格式
  let content: MessageContentItem[] = [];
  if (typeof m.content === "string") {
    content = [{ type: "text", text: m.content }];
  } else if (Array.isArray(m.content)) {
    content = m.content.map((item) => ({
      type: item.type || "text",
      text: item.text, name: item.name, args: item.args || item.arguments,
    }));
  } else if (typeof m.text === "string") {
    content = [{ type: "text", text: m.text }];
  }

  return { role, content, timestamp, id };
}
```

**消息分组**将连续同角色的消息合并为一个 `MessageGroup`，渲染时共享头像和时间戳：

```
用户消息 A ─┐
用户消息 B ─┼─→ MessageGroup { role: "user", messages: [A, B] }
            │
助手消息 C ─┼─→ MessageGroup { role: "assistant", messages: [C] }
```

# 消息发送流程

发送一条消息的完整流程涉及三个层次：

```
handleSendChat()            ← UI 层：草稿管理、命令识别、队列
  └─ sendChatMessageNow()   ← 控制层：乐观更新、工具流重置
       └─ sendChatMessage() ← 网络层：RPC 调用 chat.send
```

**第一层：命令识别与队列**

```
// ui/src/ui/app-chat.ts
export async function handleSendChat(host: ChatHost, messageOverride?: string) {
  const message = (messageOverride ?? host.chatMessage).trim();

  // 停止命令（stop/esc/abort/wait/exit 或 /stop）
  if (isChatStopCommand(message)) {
    await handleAbortChat(host);
    return;
  }

  // 新会话命令（/new 或 /reset）
  const refreshSessions = isChatResetCommand(message);

  // 如果正在忙（已有运行中的对话），加入队列
  if (isChatBusy(host)) {
    enqueueChatMessage(host, message, attachments, refreshSessions);
    return;
  }

  await sendChatMessageNow(host, message, { ... });
}
```

消息队列机制让用户可以在助手回复过程中继续输入。队列中的消息在当前轮次完成后自动逐条发送：

```
// 轮次结束时自动刷新队列
async function flushChatQueue(host: ChatHost) {
  if (!host.connected || isChatBusy(host)) return;
  const [next, ...rest] = host.chatQueue;
  if (!next) return;
  host.chatQueue = rest;
  const ok = await sendChatMessageNow(host, next.text, { ... });
  if (!ok) host.chatQueue = [next, ...host.chatQueue]; // 失败则放回
}
```

> **衍生解释 — 乐观更新（Optimistic Update）**
>
> 在网络应用中，"乐观更新"是指在服务端确认之前就先更新 UI。用户发送消息时，消息立即显示在聊天列表中（而非等待 Gateway 响应），如果发送失败再回滚。这让界面感觉更流畅，消除了网络延迟带来的"卡顿感"。

### 第二层：乐观更新与 RPC 调用

```
// ui/src/ui/controllers/chat.ts
export async function sendChatMessage(
  state: ChatState,
  message: string,
  attachments?: ChatAttachment[],
): Promise<string | null> {
  // 乐观更新：立即将用户消息加入列表
  state.chatMessages = [...state.chatMessages, {
    role: "user",
    content: contentBlocks,  // 文本 + 图片
    timestamp: Date.now(),
  }];

  // 生成运行 ID，进入流式接收状态
  const runId = generateUUID();
  state.chatRunId = runId;
  state.chatStream = "";
  state.chatStreamStartedAt = Date.now();

  try {
    await state.client.request("chat.send", {
      sessionKey: state.sessionKey,
      message: msg,
      deliver: false,          // 不通过其他渠道投递
      idempotencyKey: runId,   // 幂等键，防止重复发送
      attachments: apiAttachments,
    });
    return runId;
  } catch (err) {
    // 发送失败：清空运行状态，显示错误消息
    state.chatRunId = null;
    state.chatStream = null;
    state.chatMessages = [...state.chatMessages, {
      role: "assistant",
      content: [{ type: "text", text: "Error: " + error }],
    }];
    return null;
  }
}
```

关键参数解析：

- `deliver: false` ——WebChat 的消息只在 Gateway 内部处理，不投递到其他渠道
- `idempotencyKey: runId` ——如果网络抖动导致请求重传，Gateway 可以用此键去重

## 图片附件

WebChat 支持通过粘贴（Ctrl+V）添加图片附件：

```
// ui/src/ui/views/chat.ts
function handlePaste(e: ClipboardEvent, props: ChatProps) {
  const items = e.clipboardData?.items;
  for (const item of items) {
    if (item.type.startsWith("image/")) {
      e.preventDefault();
      const file = item.getAsFile();
      const reader = new FileReader();
      reader.addEventListener("load", () => {
        const dataUrl = reader.result as string;
        props.onAttachmentsChange([...current, {
          id: generateAttachmentId(),
          dataUrl,
          mimeType: file.type,
        }]);
      });
      reader.readAsDataURL(file);
    }
  }
}
```

图片以 Data URL（Base64 编码）形式暂存在客户端。发送时转换为 API 格式：

```
const apiAttachments = attachments.map(att => {
  const parsed = dataUrlToBase64(att.dataUrl);  // 解析 
  return { type: "image", mimeType: parsed.mimeType, content: parsed.content };
});
```

## 中止与命令

用户可以通过多种方式中止正在进行的对话：

| 方式 | 触发 |
| --- | --- |
| 输入 `stop` / `abort` / `esc` / `wait` / `exit` | `isChatStopCommand` 识别 |
| 输入 `/stop` | 命令前缀 |
| 点击 "Stop" 按钮 | `canAbort && chatRunId` |

中止操作调用 `chat.abort` RPC：

```
export async function abortChatRun(state: ChatState): Promise<boolean> {
  await state.client.request("chat.abort", {
    sessionKey: state.sessionKey,
    runId: state.chatRunId,
  });
  return true;
}
```

## 聊天视图布局

`renderChat` 函数组合了整个聊天界面的布局：

```
┌──────────────────────────────────────────┐
│  [Error callout / Compaction indicator]  │
│┌────────────────────┬────────────────────┐│
└│                    │                    │└
 │  chat-thread       │  sidebar           │
 │  (消息列表)         │  (工具输出预览)     │
 │                    │                    │
 ├────────────────────┴────────────────────┤
 │  [Queue display: Queued (N)]             │
 │  [New messages ↓ button]                 │
 │  ┌──────────────────────────────────────┐│
 │  │ textarea: Message (↵ send, Shift+↵   │
 │  │           line break, paste images)  │
 │  │ [attachments]  [Stop/New] [Send ↵]   │
 │  └──────────────────────────────────────┘│
 └──────────────────────────────────────────┘
```

聊天线程和侧边栏之间通过可拖拽分隔器 `<resizable-divider>` 分割，比例保存在 `splitRatio` 中（默认 0.6，范围 0.4-0.7）。侧边栏用于展示工具调用的输出内容——点击聊天中的工具卡片即可在侧边栏中查看完整输出。

聊天线程内的渲染使用 Lit 的 `repeat()` 指令进行高效列表更新——相比简单的 `map()`，`repeat()` 基于 key 进行 DOM 复用，避免在消息追加时重新创建所有已有消息的 DOM 节点。

## 流式输出展示

当助手正在回复时，UI 会根据 `chatStream` 的状态显示不同的指示器：

```
if (props.stream !== null) {
  if (props.stream.trim().length > 0) {
    // 有内容：渲染流式消息气泡
    items.push({ kind: "stream", text: props.stream, startedAt });
  } else {
    // 空内容：显示"正在思考"的三点动画
    items.push({ kind: "reading-indicator", key });
  }
}
```

“正在思考”指示器是三个跳动的点（CSS 动画），给用户一个视觉反馈表明助手正在处理。一旦第一个 delta 到达，指示器被替换为实际的流式文本。

# 本节小结

1. **WebChat 是 Gateway WebSocket 协议的一个客户端**，以 `operator` 角色连接，拥有完整管理权限；会话通过 sessionKey 标识和切换。
2. **事件驱动更新**：消息发送后通过 `chat` 事件接收流式 delta、最终结果、中止和错误通知，delta 采用全量替换模式防止乱序。
3. **消息发送三层架构**：UI 层处理命令识别和消息队列、控制层执行乐观更新和工具流重置、网络层发起 `chat.send` RPC 调用。
4. **消息渲染管线**：原始消息 → 规范化（统一格式）→ 分组（连续同角色合并）→ Lit 模板渲染（Markdown + 头像 + 工具卡片）。
5. **消息队列**允许用户在助手回复过程中继续输入，队列消息在当前轮次完成后自动发送。
6. **图片附件**通过剪贴板粘贴添加，以 Base64 Data URL 形式暂存，发送时转换为 API 格式。